

# Bases de Datos - SQL

---

23/04/2026

- Almacenamiento organizado de información.
- Facilitan la búsqueda, recuperación y gestión de datos.
- Son esenciales en sistemas de información, aplicaciones web, móviles y empresariales.

---

## ¿Qué son las Bases de Datos?

- Década de 1960: aparecen los primeros sistemas para gestionar datos.
- Década de 1970: surge el modelo relacional.
- Luego se consolida SQL como lenguaje estándar.

---

# Historia

- Archivos planos (CSV, TXT).
- Hojas de cálculo (Excel).
- Bases de datos no relacionales (NoSQL).

---

¿Existen otras opciones?

- Base de datos: conjunto de datos y objetos, como tablas, vistas, índices y relaciones.
  - Sistema de base de datos: software que permite crear, administrar y consultar la base de datos.
  - Cliente o herramienta de acceso: programa desde el cual los usuarios se conectan y trabajan con la base de datos.
- 

## Elementos de un sistema de base de datos

- PostgreSQL: open source, potente y flexible.
- MySQL: open source, muy utilizado.
- SQL Server: desarrollado por Microsoft, frecuente en entornos empresariales.
- Oracle Database: muy usado en sistemas corporativos.

---

## Sistemas de base de datos

- DBeaver: cliente multiplataforma con interfaz gráfica.
- pgAdmin: herramienta orientada a PostgreSQL.
- MySQL Workbench: herramienta orientada a MySQL.
- Terminal / línea de comandos: acceso textual para administrar y consultar.

---

## Cientes o herramientas de acceso

- SQL (Structured Query Language) es el lenguaje estándar para gestionar bases de datos relacionales.
- **Permite:**
  - Definir estructuras.
  - Manipular datos.
  - Consultar información.

---

## ¿Qué es y para qué sirve SQL?



- SELECT: Extrae datos de una o más tablas.
- FROM: Especifica la tabla de donde se extraerán los datos.
- WHERE: Filtra los resultados según una condición.

```
SELECT nombre, apellido FROM clientes WHERE  
ciudad = 'Madrid';
```

---

## Estructura del SELECT - 1

- ORDER BY (asc o desc): Permite ordenar los resultados por una o más columnas y ascendente o descendientemente.

```
SELECT nombre, apellido FROM clientes WHERE  
ciudad = 'Madrid' ORDER BY apellido DESC;
```

---

## Estructura del SELECT - 2

- LIMIT: Permite limitar la cantidad de resultados mostrados.

```
SELECT nombre, apellido FROM clientes WHERE  
ciudad = 'Madrid' ORDER BY apellido DESC  
LIMIT 10;
```

---

## Estructura del SELECT - 3

- CREATE DATABASE: Crea una nueva base de datos.  
`CREATE DATABASE mi_base_de_datos;`
- DROP DATABASE: Elimina una base de datos existente.  
`DROP DATABASE mi_base_de_datos;`  
CUIDADO! Es un comando destructivo!

---

## Comandos básicos de SQL - 1

- CREATE TABLE: Crea una nueva tabla.  
`CREATE TABLE clientes (id INT, nombre VARCHAR(50), apellido VARCHAR(50));`
  - DROP TABLE: Elimina una tabla existente.  
`DROP TABLE clientes;`  
CUIDADO! Es un comando destructivo!
- 

## Comandos básicos de SQL - 2

- INSERT INTO: Inserta nuevos registros en una tabla.  
`INSERT INTO clientes (id, nombre, apellido)  
VALUES (1, 'Juan', 'Pérez');`
- DELETE FROM: Elimina registros de una tabla.  
`DELETE FROM clientes WHERE id = 1;`

---

## Comandos básicos de SQL - 3

- UPDATE: Modifica los valores de registros existentes.  
UPDATE clientes SET apellido = 'García' WHERE  
id = 2;

---

## Comandos básicos de SQL - 4

- `docker compose up -d`
- `docker ps`
- `docker exec -it clase_sql-db-1 /bin/bash`
- `psql -U postgres`
  
- `docker exec -it clase_sql-db-1 psql -U postgres -d nuestra_db`

---

## Uso de PostgreSQL con Docker



- PostgreSQL, tutorial SQL oficial:  
<https://www.postgresql.org/docs/18/tutorial-sql-intro.html>
- PostgreSQL, CREATE TABLE:  
<https://www.postgresql.org/docs/18/tutorial-table.html>
- PostgreSQL, SELECT:  
<https://www.postgresql.org/docs/current/static/sql-select.html>
- W3Schools: <https://www.w3schools.com/sql/>

---

## Referencias

- NULL: indica que una columna puede no tener valor.
- NOT NULL: obliga a que la columna tenga un valor.
- UNIQUE: evita valores repetidos en una columna.
- Estas restricciones ayudan a mantener la calidad y consistencia de los datos.

---

## Restricciones básicas de columnas

Ejemplos:

```
nombre VARCHAR(50) NOT NULL,  
email VARCHAR(100) UNIQUE,  
telefono VARCHAR(20) NULL
```

---

## Restricciones básicas de columnas

- Primary Key (clave primaria): identifica de manera única cada registro de una tabla.
- Foreign Key (clave foránea): campo que referencia la clave primaria de otra tabla.
- Las claves foráneas permiten relacionar tablas.

---

## Claves primarias y claves foráneas

- Un registro de una tabla se relaciona con un solo registro de otra tabla.
- Se usa cuando dos entidades tienen correspondencia única.

Ejemplo: persona - pasaporte

- Una persona tiene un pasaporte y un pasaporte pertenece a una sola persona.

---

## Relación uno a uno (1:1)

- Un registro de una tabla puede relacionarse con muchos registros de otra.
- Es la relación más común en bases de datos relacionales.

Ejemplo: cliente - pedido

- Un cliente puede tener muchos pedidos y cada pedido pertenece a un solo cliente.

---

## Relación uno a muchos (1:N)

- Un registro de una tabla puede relacionarse con muchos de otra tabla, y viceversa.
- Se implementa mediante una tabla intermedia.

Ejemplo: alumno - curso

- Un alumno puede inscribirse en varios cursos y un curso puede tener varios alumnos.
- Implementación: tabla intermedia -> inscripciones(id\_alumno, id\_curso)

---

## Relación muchos a muchos (N:M)

- PostgreSQL Tutorial, Foreign Keys: <https://www.postgresql.org/docs/current/tutorial-fk.html>
- PostgreSQL CREATE TABLE: <https://www.postgresql.org/docs/current/sql-createtable.html>
- IBM, Database Relationships: <https://www.ibm.com/docs/en/masv-and-l/maximo-manage/8.3.0?topic=structure-database-relationships>
- Vertabelo, One-to-One: <https://vertabelo.com/blog/one-to-one-relationship-in-database/>
- Vertabelo, One-to-Many: <https://vertabelo.com/blog/one-to-many-relationship/>
- Vertabelo, Many-to-Many: <https://vertabelo.com/blog/many-to-many-relationship/>

---

# Referencias